

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.4

Lists, Tuples & Sets

Day 4 — storing many values in one place

90 min

Concepts + live code

30 min

Practice notebook

3

Collection types

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 3

1 What does a function with no return give back?

→ None — automatically.

2 print or return: which does the caller get?

→ Only return. print just displays.

3 Why is `def f(x=[])` dangerous?

→ The default list is created once and shared by every call.

4 What does LEGB stand for?

→ Local, Enclosing, Global, Built-in.

5 Why does `count = count + 1` fail inside a function?

→ Assigning makes count local for the whole body.

Question 3 is today's headline act. The reason behind it is the reason this whole session exists.

How today's two hours are structured

1

Lists

Indexing, slicing, methods, sorting, copying

35 min

2

Tuples

Immutability, unpacking, when to prefer them

15 min

3

Sets

Uniqueness, set algebra, fast membership

20 min

4

Choosing between them

A decision table and conversions

10 min

5

Mini build

Cleaning a messy list of student records

10 min

6

Practice notebook

You code, I walk around

30 min

Lists get half the session because you will use them a hundred times more often than the other two combined.

By the end of today you will be able to

1

Store

Build lists, tuples and sets, and reach any element inside them

2

Slice

Take any section of a sequence with [start:stop:step]

3

Modify

Add, remove, sort and copy — and know which of those change the original

4

Choose

Pick the right structure for a task instead of defaulting to a list

The question to keep asking today

“Does this change the original, or give me a new one?” Nearly every method today falls into one camp or the other, and mixing them up is the source of most collection bugs. Print the variable after every operation until the distinction is automatic.

1

Lists

Ordered, changeable, duplicates allowed · 35 minutes

- Creating & indexing
- Slicing
- Methods
- Sorting
- Copying & aliasing

Creating a list, and reaching inside it

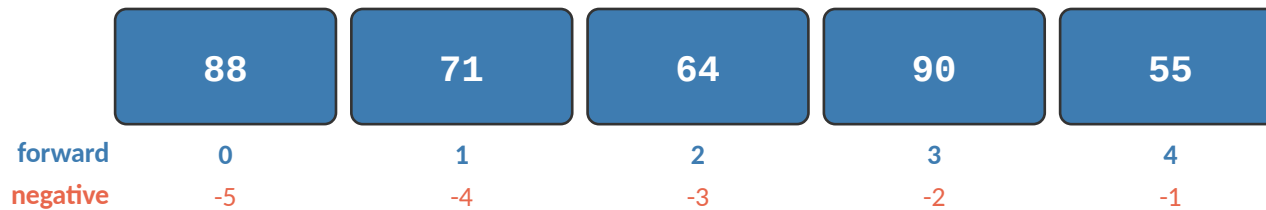
A list holds many values in order

```
marks = [88, 71, 64, 90, 55]
mixed = [1, "two", 3.0, True] # any types
empty = []
nested = [[1, 2], [3, 4]]     # lists in lists
chars = list("PYTHON")        # from a string
```

Three things every list gives you

- Order — position 0 stays position 0
- Duplicates — the same value can appear many times
- Mutability — you can change it after creating it

Positions run forwards from 0 and backwards from -1

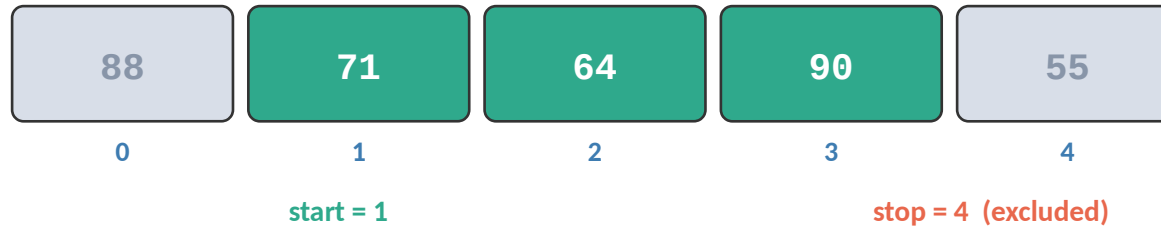


`marks[-1]` is the safe way to get the last item — it never needs `len()`.

```
marks[0]    -> 88
marks[-1]   -> 55    # last, always
marks[2]    -> 64
len(marks)  -> 5
marks[5]    -> IndexError
```

Slicing — taking a section

marks[1:4] — from position 1, up to but NOT including 4



```
marks[1:4] -> [71, 64, 90]
marks[:3]  -> [88, 71, 64]
marks[2:]  -> [64, 90, 55]
marks[:]   -> a full COPY
```

The third number is the step

```
marks[::2] -> [88, 64, 55]    every 2nd
marks[1::2] -> [71, 90]       from 1, every 2nd
marks[::-1] -> [55, 90, 64, 71, 88]
#           ^ reverses the whole list
"hello"[::-1] -> 'olleh'      works on strings
```

Slicing never raises IndexError

```
marks[10]      -> IndexError
marks[2:99]    -> [64, 90, 55]    no error
marks[99:]     -> []              no error
```

Convenient, but it hides mistakes: an empty result may mean your indexes were wrong, not that the data was empty.

List methods — adding and removing

Method	What it does	Example on [1, 2, 3]	Result
<code>append(x)</code>	add one item at the end	<code>nums.append(4)</code>	<code>[1, 2, 3, 4]</code>
<code>insert(i, x)</code>	add at position i	<code>nums.insert(0, 9)</code>	<code>[9, 1, 2, 3]</code>
<code>extend(other)</code>	add ALL items of another list	<code>nums.extend([4, 5])</code>	<code>[1, 2, 3, 4, 5]</code>
<code>remove(x)</code>	delete the first matching VALUE	<code>nums.remove(2)</code>	<code>[1, 3]</code>
<code>pop(i)</code>	remove by POSITION and return it	<code>nums.pop(0)</code>	<code>returns 1</code>
<code>clear()</code>	empty the list	<code>nums.clear()</code>	<code>[]</code>

⚠ append vs extend

```
[1,2].append([3,4]) -> [1, 2, [3, 4]]    nested!  
[1,2].extend([3,4]) -> [1, 2, 3, 4]      flat
```

These all change the list IN PLACE

They return None, not a new list. Writing `nums = nums.append(4)` destroys your list — `nums` becomes None.

Sorting — the clearest example of the day's question

`nums.sort()` — changes the original

```
nums = [3, 1, 2]
nums.sort()
print(nums)          # [1, 2, 3]  changed

result = nums.sort()
print(result)         # None !

# It sorts in place and returns nothing.
```

`sorted(nums)` — gives a new list

```
nums = [3, 1, 2]
result = sorted(nums)
print(result)         # [1, 2, 3]
print(nums)           # [3, 1, 2]  untouched

# Works on ANY iterable:
sorted("hello")        # ['e', 'h', 'l', 'l', 'o']
sorted({3, 1, 2})      # [1, 2, 3]
```

Both take the same two options

```
nums.sort(reverse=True)      # largest first

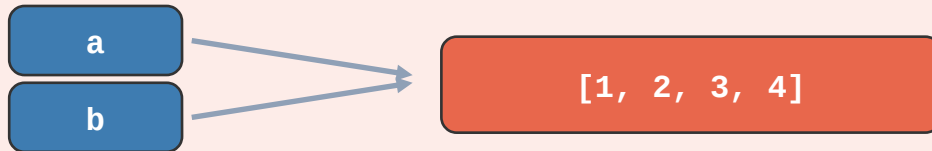
students.sort(key=lambda s: s[1])  # by score
```

The `key=` option takes a function that says what to sort BY — Day 3's lambda, doing the job it exists for. `sorted()` is usually the safer default: if you keep the original you can always sort again, but an in-place sort has thrown the original order away for good.

Copying a list — and the trap you have met three times

! b = a does NOT copy

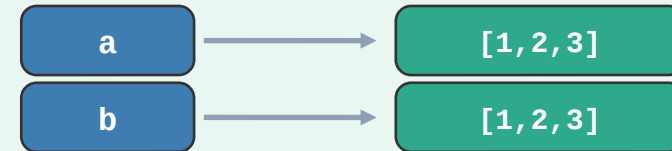
```
a = [1, 2, 3]
b = a          # a SECOND NAME, not a copy
b.append(4)
print(a)       # [1, 2, 3, 4] a changed too
```



one object

Three ways to make a real copy

```
b = a[:]      # slice the whole thing
b = a.copy()  # clearest to read
b = list(a)   # rebuild it
```



two separate objects

This is the fourth time you have met this idea

Day 1: `c = a`, then `c.append()` changed `a`. Day 3: the mutable default argument that kept growing. Day 3: a function that appended to the caller's list. Today: `b = a`. Every one of them is the same sentence — two names, one object. It is not four rules to memorise. It is one rule you have now seen from four angles.



Tuples

Ordered, but locked · 15 minutes

- Creating
- Immutability
- Unpacking
- When to prefer them

Tuples — a list that cannot be changed

Round brackets instead of square

```
point = (3, 7)
rgb = (255, 128, 0)
single = (5,)      # trailing comma REQUIRED
not_a_tuple = (5)   # this is just the int 5

point[0] -> 3        indexing works
point[0:1] -> (3,)   slicing works
```

⚠ Everything that changes it fails

```
point[0] = 9
-> TypeError: 'tuple' object does
    not support item assignment

point.append(9)
-> AttributeError: no attribute
    'append'
```

Why lock a collection on purpose?

- It signals intent: this is not meant to change
- It cannot be corrupted by accident from elsewhere
- It can be a dictionary key; a list cannot

You have used tuples already

```
p, q = q, p          # Day 1 swap
return lo, hi, avg    # Day 3 multi-return
for i, v in enumerate(x) # Day 2
def f(*args)          # args IS a tuple
```

Unpacking, and one thing immutability does not protect

Unpacking works on any sequence

```
x, y = (3, 7)
a, b, c = [1, 2, 3]      # lists too

first, *rest = [1, 2, 3, 4]
# first -> 1      rest -> [2, 3, 4]

a, b = (1, 2, 3)    -> ValueError: too many
```

⚠ Immutability does not mean frozen inside

```
t = (1, [2, 3])

t[1] = [9]      -> TypeError
t[1].append(4)  -> works!

print(t)        # (1, [2, 3, 4])
# The TUPLE is locked. The list inside is not.
```

The distinction, stated once

A tuple guarantees that its slots always point at the same objects. It says nothing about whether those objects can themselves change. That is why a tuple containing a list cannot be used as a dictionary key — Python cannot rely on it staying the same.

If you need a genuinely fixed collection, keep only immutable things inside it: numbers, strings, and other tuples.

3

Sets

Unique, unordered, and fast · 20 minutes

- Uniqueness
- No order, no index
- Set algebra
- Fast membership

Sets — every value appears exactly once

Curly brackets, duplicates dropped

```
s = {3, 1, 2, 3, 1}
print(s)           # {1, 2, 3}    duplicates gone

empty = set()      # NOT {} - that is a dict!

s.add(4)           s.discard(9)    # no error
s.remove(9)        -> KeyError if missing
```

⚠ No order means no indexing

```
s[0]
-> TypeError: 'set' object is not
    subscriptable

# No slicing, no .sort(), no positions.
# You can loop over it and test membership,
# but you cannot ask for 'the first one'.
```

The everyday use: removing duplicates

```
names = ["Ravi", "Sara", "Ravi", "Amit"]
unique = list(set(names))    # order LOST

# Order kept (Python 3.7+):
unique = list(dict.fromkeys(names))
```

Membership is dramatically faster

Checking `x` in `my_list` scans every item until it finds a match. Checking `x` in `my_set` jumps straight to the answer, however large the set is. On a million items that is the difference between noticeable and instant.

Set algebra — combining two sets



Union



only a

both

only b

shaded = in the result

everything in either

```
a | b
{1, 2, 3, 4, 5}
```



Intersection



only a

both

only b

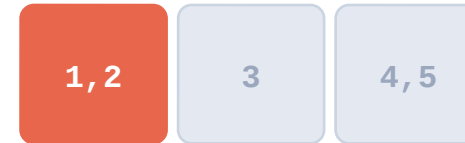
shaded = in the result

only what is in both

```
a & b
{3}
```



Difference



only a

both

only b

shaded = in the result

in a but not in b

```
a - b
{1, 2}
```



Symmetric diff



only a

both

only b

shaded = in the result

in one but not both

```
a ^ b
{1, 2, 4, 5}
```

a = {1, 2, 3} **b = {3, 4, 5}**

methods read better in shared code.

Each operator has a named method too: `a.union(b)`, `a.intersection(b)`, `a.difference(b)`. The



Choosing

Which structure, and how to convert · 10 minutes

- Decision table
- Conversions
- Common patterns

List, tuple or set?

	list	tuple	set
Written as	[1, 2, 3]	(1, 2, 3)	{1, 2, 3}
Keeps order	yes	yes	no
Can be changed	yes	no	yes (add / remove)
Allows duplicates	yes	yes	no
Index and slice	yes	yes	no
Can be a dict key	no	yes*	no
Membership test	slow on big data	slow on big data	fast, any size

* only if everything inside it is immutable too

 **Use a list**
for almost everything — ordered data you will change

 **Use a tuple**
for fixed records: coordinates, RGB, a row of results

 **Use a set**
for uniqueness or fast membership checks

Converting between them, and three patterns worth memorising

Conversions

```
list((1, 2, 3))    -> [1, 2, 3]
tuple([1, 2, 3])   -> (1, 2, 3)
set([1, 2, 2])     -> {1, 2}
list({3, 1, 2})    -> [1, 2, 3]  order not
                        guaranteed
list("abc")        -> ['a', 'b', 'c']
```

A caution on set conversions

`list(set(x))` is the quick way to deduplicate, but it silently throws away the original order. If order matters — and with rows of data it almost always does — use `dict.fromkeys()` instead. This has quietly reordered a great many student datasets.

1 Remove duplicates, keep order

```
names = ["Ravi", "Sara", "Ravi"]
list(dict.fromkeys(names))
# ['Ravi', 'Sara']
```

2 Fast membership filter

```
valid = set(valid_ids)  # build once
for r in rows:
    if r.id in valid: ...  # fast
```

3 What is missing from a list?

```
expected = {"Python", "Stats", "ML"}
done      = {"Python"}
expected - done  # {'Stats', 'ML'}
```

Debug corner — eight collection bugs

You wrote	What happens	Fix
<code>nums = nums.append(4)</code>	nums becomes None	<code>nums.append(4)</code>
<code>b = a</code>	both names change together	<code>b = a.copy()</code>
<code>nums = nums.sort()</code>	nums becomes None	<code>nums.sort()</code> or <code>sorted(nums)</code>
<code>[1,2].append([3,4])</code>	gives a nested list	<code>extend([3, 4])</code>
<code>empty = {}</code>	that is a dict, not a set	<code>empty = set()</code>
<code>my_set[0]</code>	TypeError, sets have no order	loop over it, or convert to a list
<code>single = (5)</code>	that is an int, not a tuple	<code>single = (5,)</code>
<code>list(set(rows))</code>	order silently lost	<code>list(dict.fromkeys(rows))</code>

Rows 1 and 3 are the same mistake: in-place methods return None. Row 2 is the aliasing thread. All eight are in today's practice notebook.

Putting it together — cleaning a messy attendance list

```
raw = ["Ravi", "sara", "RAVI", "Amit", "Sara", "", "Priya", "Kiran"]
enrolled = {"ravi", "sara", "amit", "priya", "neha"}

def clean(names):
    """Normalise names and drop blanks, keeping order."""
    out = []
    for n in names:
        n = n.strip().lower()      # tidy the text
        if n:                     # skip empty strings
            out.append(n)
    return out

present = clean(raw)
unique = list(dict.fromkeys(present)) # dedupe, keep order

absent = enrolled - set(unique)      # set difference
unknown = set(unique) - enrolled     # not on the roll

print("Present :", unique)
print("Absent  :", sorted(absent))
print("Unknown :", sorted(unknown))
```



List

raw and present keep order



Set

enrolled: uniqueness + fast lookup



dict.fromkeys

dedupe WITHOUT losing order



Set difference

who is missing, in one line



sorted()

a new list, roll left untouched



Day 3's function

one job, docstring, returns

Doing this with lists alone would need a nested loop. The set does it in one line.

Practice session — 30 minutes



Predict the output

Slicing, in-place methods, aliasing

8 questions



Spot & fix the bug

None returns, b = a, empty set, tuple of one

4 questions



Write the code

Slicing, dedupe, top-3, common items, matrix

5 tasks

Your notebook

[Day4_Practice_Questions.ipynb](#)

- Print the list after every step
- Check: changed, or new?
- Raise a hand — I am walking around
- Solutions shared after class

If a variable mysteriously becomes None, you assigned the result of an in-place method. That is today's single most likely bug.

TODAY IN TWELVE LINES

Recap

- 1 list `[]` ordered and changeable; tuple `()` locked; set `{ }` unique.
- 2 Index from 0 forwards, from -1 backwards.
- 3 Slices exclude the stop, exactly like `range()`.
- 4 `[::-1]` reverses; `[:]` makes a copy.
- 5 `append` adds one item; `extend` adds all of them.
- 6 In-place methods return `None` — never reassign them.
- 7 `sort()` changes the list; `sorted()` returns a new one.
- 8 `b = a` is a second name. `b = a.copy()` is a copy.
- 9 A single-item tuple needs the trailing comma: `(5,)`
- 10 A tuple is locked, but a list inside it is not.
- 11 `set()` for an empty set — `{ }` is an empty dict.
- 12 Sets: no order, no index, no duplicates, fast lookup.

Homework

Write `remove_duplicates(items)` keeping order · Given two lists, print items in both, in either, and in only one · Rotate a list by `n` positions using slicing

Next class — Topic 1.5

Dictionaries: key-value pairs, nested dicts, dictionary methods, and iterating over them